

Enhancing Reverse Engineering with AI

Data and Network Security

Edoardo Allegrini

edoardoallegrini.github.io



allegrini@di.uniroma1.it

More than a Title



→
[Oct 2020 – Jun 2023]

B.Sc. in Computer Science



→
[Oct 2023 – Oct 2025]

M.Sc. in Cybersecurity



→
[Nov 2025 – Present]

Ph.D. in Cybersecurity



Active Investigations & Technical Background



→
[Jan 2024 – Jan 2025]

Research Scholarship

Social Network Anomaly Detection



→
[Mar 2025 – May 2025]

Research Scholarship

Reverse Engineering & AI



→
[Sep 2025 – Dec 2025]

Research Collaborator Contract

Social Agents and Disinformation



→
[Mar 2025 – Present]

Research Collaborator

Agentic AI Security



→
[Nov 2025 – Present]

Ph.D. Collaboration

Reverse Engineering & AI



Key Contributions

Formalizing the Safety, Security, and Functional Properties of Agentic AI Systems

2026

Edoardo Allegrini, Ananth Shreekumar, Z. Berkay Celik ▪ *ICLR Workshop on Agents in the Wild: Safety, Security, and Beyond*

BotVerse: Real-Time Event-Driven Simulation of Social Agents (Demo)

2026

Edoardo Allegrini, Edoardo Di Paolo, Angelo Spognardi, Marinella Petrocchi ▪ *Int. Conf. on Autonomous Agents and Multiagent Systems*

Simulating Social Media with LLM-Powered Agents: Demography, Psychography, and Disinformation Dynamics

2026

Edoardo Allegrini, Edoardo Di Paolo, Angelo Spognardi, Marinella Petrocchi ▪ *Workshop on Multi-Agent-Based Simulation (at AAMAS)*

Deciphering Social Behaviour: a Novel Biological Approach For Social Users Classification

2025

Edoardo Allegrini, Edoardo Di Paolo, Marinella Petrocchi, Angelo Spognardi ▪ *ACM/SIGAPP Symposium on Applied Computing (SAC)*

A Proposal for Uncovering Hidden Social Bots via Genetic Similarity

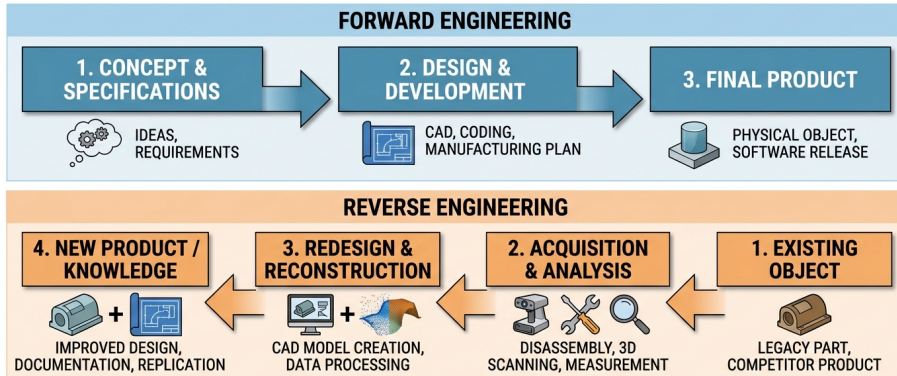
2024

Edoardo Allegrini, Edoardo Di Paolo, Marinella Petrocchi, Angelo Spognardi ▪ *Int. Conf. on Discovery Science (DS)*

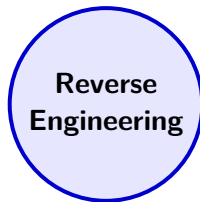
The Art of Deconstruction

What is Reverse Engineering (RE)?

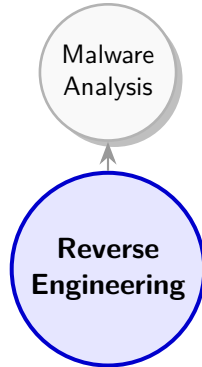
The process of extracting design, architecture, and logic from a finished product.



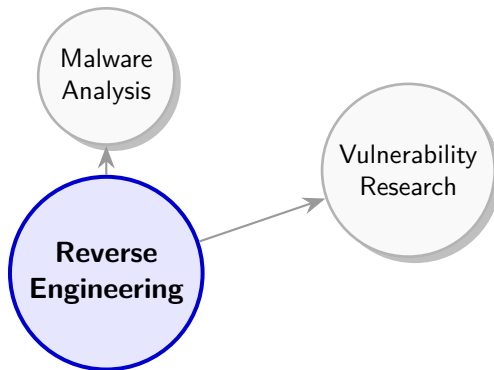
The Domains of Reverse Engineering



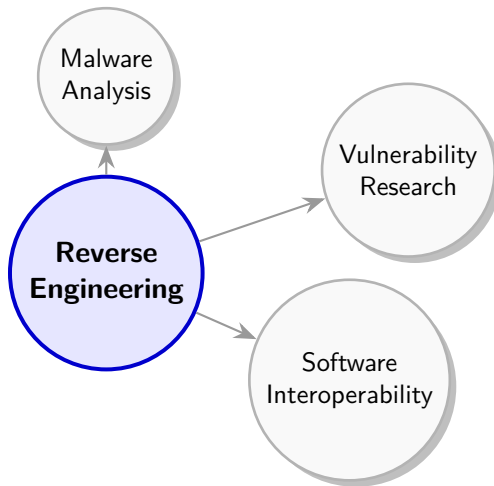
The Domains of Reverse Engineering



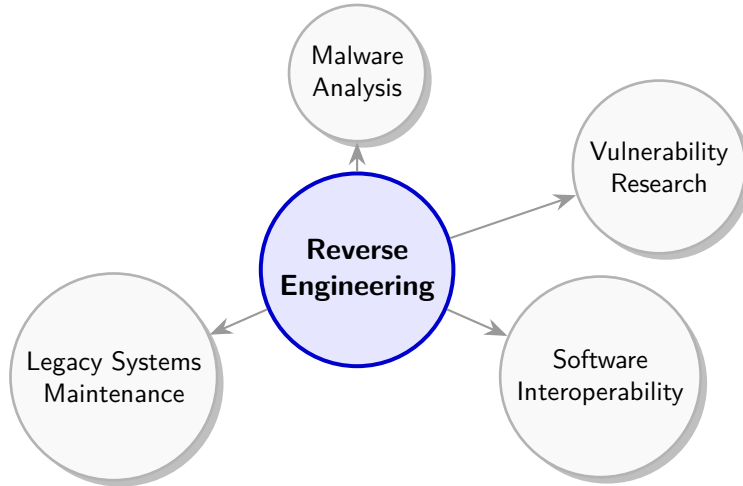
The Domains of Reverse Engineering



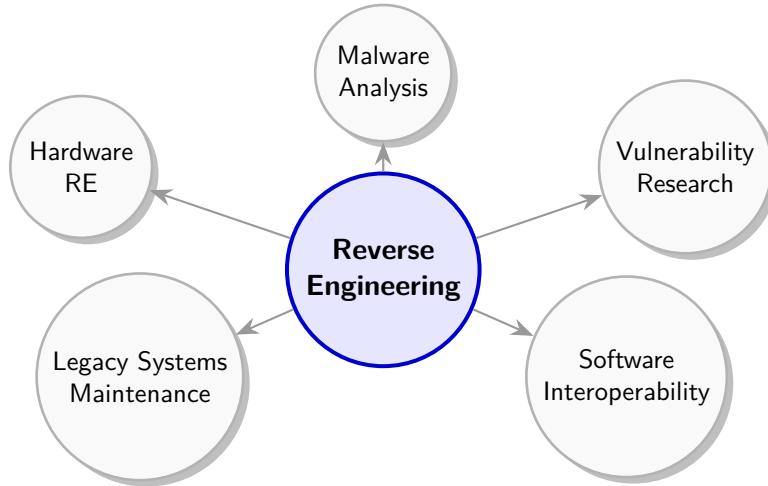
The Domains of Reverse Engineering



The Domains of Reverse Engineering



The Domains of Reverse Engineering



A Brief History: Physical to Digital



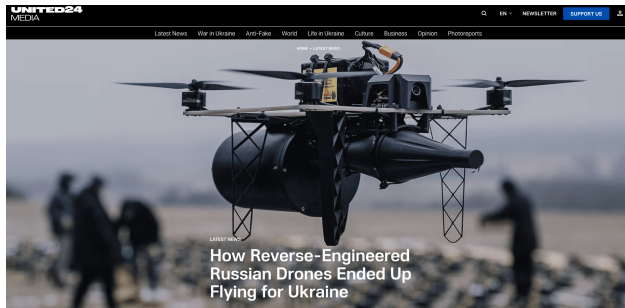
Cryptanalysis of the Enigma machine

Origins in Warfare and Industry

- RE is not a digital invention.
- **WWII Intelligence:** Capturing and analyzing enemy technology.
- **The Advent of Software (1980s):** As software became proprietary.

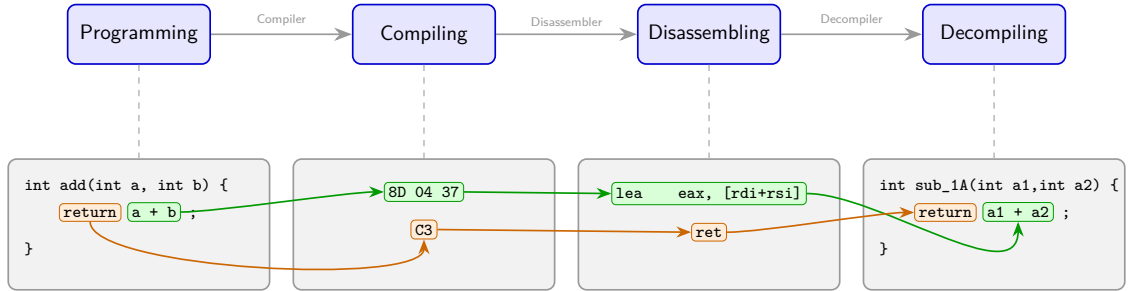
The Modern Landscape: A Physical-Digital Mix

Capturing modern tech, extracting its firmware, and reverse-engineering the code to repurpose it.



<https://united24media.com/latest-news/how-reverse-engineered-russian-drones-ended-up-flying-for-ukraine-15318>

The Executable Lifecycle



Under the Hood: The Decompilation Pipeline

From Bytes to C-Like Code (Ghidra / IDA Pro)

1. **Parsing & Disassembly:** Raw binary (e.g., 55 48 89 e5) is mapped to CPU instructions (e.g., `push rbp; mov rbp, rsp`).
2. **Intermediate Representation (IR):** Assembly is lifted into an architecture-agnostic language (e.g., P-Code in Ghidra).
3. **Data Flow Analysis:** Tracking how values are passed between registers and memory addresses over time.
4. **Control Flow Analysis:** Grouping blocks of code to reconstruct loops (`for`, `while`) and conditionals (`if/else`).
5. **Pseudo-code Generation:** The Abstract Syntax Tree (AST) is translated back into readable C-like syntax.

Why Intermediate Representation?

Disassembly (x86_64)

```
check_value:
    mov dword ptr [rbp - 4], edi
    mov eax, dword ptr [rbp - 4]
    cmp eax, 5
    jg .L2
    mov eax, 1
    jmp .L3
.L2:
    mov eax, 0
.L3:
    ret
```

Decompile
→

Decompiled Pseudo-code

```
int check_value(int a) {
    if (a > 5) {
        return 0;
    } else {
        return 1;
    }
}
```

The Magic of IR: The decompiler successfully abstracts away hardware-specific calling conventions (edi), registers (eax), stack variables, and jump offsets (.L2) into a logical if/else construct.

The Reality Check: Stripped Binaries

What is “Stripping”?

The process of removing debugging symbols: *variable names*, *function names*, and *variable types*.

Why is it so common?

- **Size Reduction**
- **Standard Practice**
- **Protection:** Raises the barrier to entry for reverse engineering.

The Analyst's Dilemma

Instead of seeing `int calculate_tax(int amount)`, the reverse engineer sees `int sub_4010A0(int a1)`. Semantic context is entirely lost!

Adversarial Tactics: Anti-RE & Obfuscation

When attackers (or paranoid software vendors) actively fight back.

Obfuscation

Disrupting human readability

- **Control Flow Flattening:** Turning clean logic into a massive, nested state machine (often called a “dispatcher”).
- **String Encryption:** Hiding URLs, IPs, or keys so they don't appear in basic static analysis.
- **Dead Code:** Inserting fake instructions to waste an analyst's time.

Anti-Analysis

Disrupting tools and environments

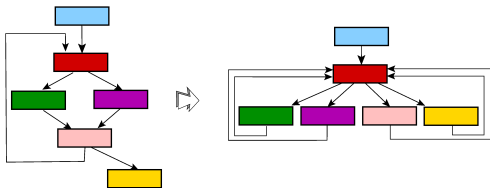
- **Anti-Debugging:** The program actively checks if a debugger is attached and crashes itself.
- **Packers:** The true executable is compressed or encrypted, only unpacking itself into memory during runtime.



Examples: CFG Flattening & Anti-Analysis

Control Flow Flattening

The “Dispatcher” in Action



Left: Original execution flow. Right: Flattened switch-loop.

Anti-Analysis (Anti-Debugging)

Hardware/VM Detection Check

```

0040184E      ; Attributes: bp-based frame
0040184E
0040184E      vmware_anti_debug proc near
0040184E
0040184E      var_1C= dword ptr -1Ch
0040184E      ms_exc= CPPEH_RECORD ptr -18h
0040184E
0040184E  6A 0C      push     0Ch
00401850  68 90 E7 40 00 push     offset stru.40E790
00401855  E8 06 96 00 00 call     __SEH_prolog
0040185A  83 65 E4 00   and     [ebp+var_1C], 0
0040185E  83 65 FC 00   and     [ebp+ms_exc.registration.TryLevel], 0
00401862  53          push     ebx
00401863  8B 68 58 40 56 mov     eax, 56405868h ; VMXh - magic number
00401868  8B 65 D4 85 86 mov     ebx, 8685D465h
0040186D  09 0A 00 00 00 mov     ecx, 0Ah ; 0A - Command to get VMware version
00401872  66 8A 58 56   mov     dx, 5658h ; 5X - VMware I/O port
00401876  ED          in       eax, dx
00401877  89 5D E4      mov     [ebp+var_1C], ebx
0040187A  5B          pop     ebx
0040187B  EB 07      jmp     short loc_401884

```

VMware I/O port "Magic" check.

The Modern Bottleneck – Motivation

- **Scale:** No need to elaborate on the increasing trend of malware spread.
- **Cognitive Load**



Question:

How can we accelerate this highly complex, pattern-heavy process?

```
unsigned __int64 __fastcall sub_401020(int a1)
{
    unsigned int v1;
    unsigned int v2;
    int v3;

    if ( a1 <= 0 )
        return 0i64;

    v1 = 1;
    v2 = 0;

    if ( a1 > 1 )
    {
        v3 = a1 - 1;
        do
        {
            v1 += v2;
            v2 = v1 - v2;
            --v3;
        }
        while ( v3 );
    }
    return v1;
}
```


The Modern Bottleneck – Motivation

- **Scale:** No need to elaborate on the increasing trend of malware spread.
- **Cognitive Load**



Question:

How can we accelerate this highly complex, pattern-heavy process?

With AI 🤖

```
unsigned __int64 __fastcall sub_401020(int a1)
{
    unsigned int v1;
    unsigned int v2;
    int v3;

    if ( a1 <= 0 )
        return 0i64;

    v1 = 1;
    v2 = 0;

    if ( a1 > 1 )
    {
        v3 = a1 - 1;
        do
        {
            v1 += v2;
            v2 = v1 - v2;
            --v3;
        }
        while ( v3 );
    }
    return v1;
}
```

1. Function Renaming

Context-aware labeling for opaque addresses.

sub_401020 → calc_fibonacci

1. Function Renaming

Context-aware labeling for opaque addresses.

`sub_401020` → `calc_fibonacci`

2. Variable Renaming

Meaningful identifiers via data flow analysis.

`v1`, `a1` → `url`, `index`

AI-Driven Decompilation Enhancements

1. Function Renaming

Context-aware labeling for opaque addresses.

`sub_401020` → `calc_fibonacci`

2. Variable Renaming

Meaningful identifiers via data flow analysis.

`v1`, `a1` → `url`, `index`

3. Type Inference

Recovery of custom structures and pointers.

`__int64` → `Node*`

AI-Driven Decompilation Enhancements

1. Function Renaming

Context-aware labeling for opaque addresses.

`sub_401020` → `calc_fibonacci`

2. Variable Renaming

Meaningful identifiers via data flow analysis.

`v1`, `a1` → `url`, `index`

3. Type Inference

Recovery of custom structures and pointers.

`__int64` → `Node*`

4. Code Simplification

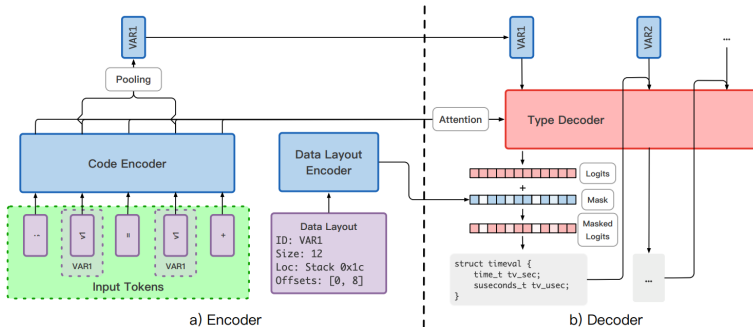
Refactoring convoluted flows to idiomatic patterns.

`do-while` / `goto` → `for` loops

The First Wave of AI

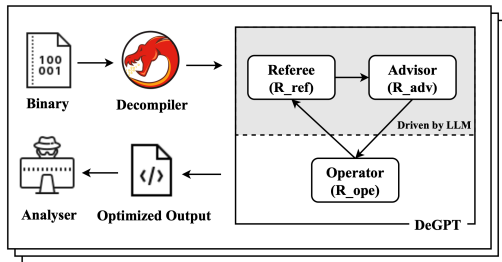
DIRTY [2] – Transformer-based **multi-classification**.

Limitations: (a) Training-bound behavior, (b) Unseen structures → incorrect output, (c) Scalability limits



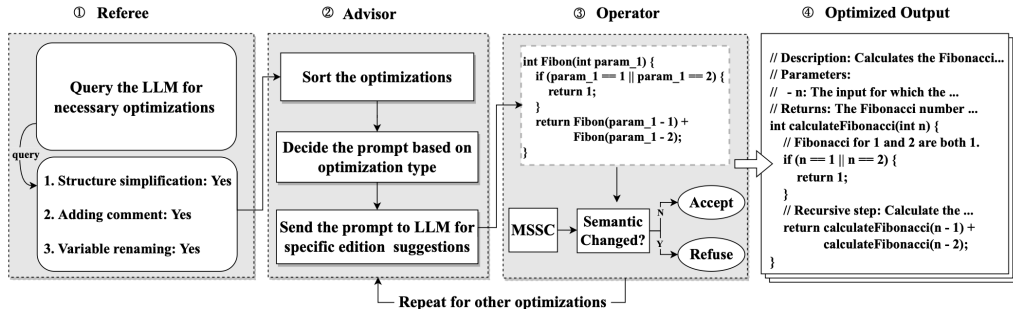
DeGPT: LLM-Powered Decompilation [3]

NDSS 2024



- ★ **Referee:** optimization scheme
- ★ **Advisor:** specific rectification measures
- ★ **Operator:** filter out the gibberish of LLM

Workflow Example



Core Innovation: The Three-Role Mechanism

To overcome the limitations of one-shot LLM usage, DeGPT employs three specialized roles to guide the LLM step-by-step:

- **Referee (R_{ref}):** Evaluates if optimizations are necessary for a snippet. *Impact:* Saves up to 21.9% in API token costs.
- **Advisor (R_{adv}):** Provides specific rectification measures and intelligently sorts optimizations (e.g., structural simplification *before* renaming).
- **Operator (R_{ope}):** The censor. Ensures the LLM's suggested changes do not alter the original function's behavior.

Fidelity Assurance: MSSC

The Challenge: LLMs are generative and can hallucinate code that breaks original logic.

Micro Snippet Semantic Calculation (MSSC): A program analysis method used by the Operator (R_{ope}) to check for behavioral equivalence.

- **Calculation Phase:** Simulates execution by assigning symbols unique random numbers and tracking value changes/invocations across execution paths.
- **Comparison Phase:** Compares the symbol tables and invocation logs of the original vs. optimized code.
- **Effectiveness:** Detected harmful changes in 21.9% of LLM simplification responses (84% accuracy, 85% recall rate).

Fidelity Assurance: MSSC

The Challenge: LLMs are generative and can hallucinate code that breaks original logic.

Micro Snippet Semantic Calculation (MSSC): A program analysis method used by the Operator (R_{ope}) to check for behavioral equivalence.

- **Calculation Phase:** Simulates execution by assigning symbols unique random numbers and tracking value changes/invocations across execution paths.
- **Comparison Phase:** Compares the symbol tables and invocation logs of the original vs. optimized code.
- **Effectiveness:** Detected harmful changes in 21.9% of LLM simplification responses (84% accuracy, 85% recall rate). **From the tests I did it doesn't work well.**

Experimental Setup: The LLM Engine

Primary Model Configuration

The core experiments utilized **ChatGPT (gpt-3.5-turbo)**, with the temperature explicitly locked at **0.2** to maintain deterministic and consistent outputs.

Alternative Exploration

- Benchmarked against **GPT-4**.
- GPT-4 showed marginal gains but offered *no overwhelming advantage*.

Dataset Details & Composition

Evaluated on **620 sets** of source code and decompiler outputs, testing both *stripped* and *non-stripped* binaries to assess symbol recovery.

Source	Sets	Domain Focus
Coreutils	310	Command-line tools
LeetCode	170	Algorithmic logic
AudioFlux	100	Audio processing library (2023)
Mirai	40	IoT Malware

Evaluation Metrics

Naming & Structural

🔑 **MVR** (*Meaningful Variable Ratio*)

Measures how closely predicted variable names match the original source names.

- Based on Levenshtein distance
- Uses synonym matching

🧠 **ER** (*Effort Ratio*)

Quantifies reduction in human cognitive effort after optimization.

- Based on Halstead's Effort metric

Commenting

✅ **CR** (*Correct Rate*)

Proportion of comments that are logically correct.

- Manually verified on a subset 😞

💡 **NR** (*Non-trivial Rate*)

Measures whether comments provide meaningful semantic insight.

- Avoids trivial restatements of code
- Manually verified on a subset 😞

Evaluation Results

Codebase	MVR (%)		ER (%)		CR (%)		NR (%)	
	①	②	①	②	①	②	①	②
LeetCode	27.0	23.0	75.5	76.9	98.7	100	64.8	36.8
Mirai	29.1	17.3	77.5	75.8	99.4	96.8	71.6	36.7
Coreutils	28.0	20.2	72.0	74.2	98.9	100	62.0	40.6
AudioFlux	37.0	36.4	77.6	78.5	99.6	100	53.0	38.4
Average	30.3	24.2	75.6	76.3	99.2	99.2	62.9	38.1

The symbol ① represents non-stripped binaries while ② represents the stripped binaries.

Validating Practical Impact: Two User Studies

To measure DeGPT's real-world effectiveness, the researchers conducted two distinct evaluations:

(1) Question-Based Study

Focus: Perceived quality and analyst attitudes.

Metric: Subjective scoring (0-10) on conciseness, naming, comments, and helpfulness.

(2) Task-Based Study

Focus: Actual performance improvements.

Metric: Quantitative measurement of accuracy and time taken to solve real reverse-engineering tasks.

Question-Based Study: Perception & Quality

Participants: 12 individuals categorized by expertise:

- **Professional:** 4+ years experience ($n = 4$)
- **Intermediate:** ~ 2 years experience ($n = 4$)
- **Introductory:** Basic RE skills ($n = 4$)

Methodology: Rated 10 pairs of raw vs. optimized outputs.

Key Insights

- **High Overall Approval:** Scores ranged from 7.0 to 9.5 across all groups.
- **Expert Skepticism** (this will come back later)
- **Conciseness Concerns:** The LLM occasionally over-explained simple statements, making the code feel slightly less idiomatic.

Task-Based Study: Performance & Efficiency

Objective: Identify the specific purpose of given code snippets.

Methodology: A/B test with 22 participants (Raw Decompiler Output vs. DeGPT-Optimized Output).

Accuracy

Raw Output: 39%
DeGPT Output: **88%**
(+126% Improvement)

Efficiency

Raw Output: 1,584s
DeGPT Output: **840s**
(47% Time Saved)

Takeaway: DeGPT lowers the barrier to entry and accelerates comprehension for reverse engineering tasks.

DeGPT: Key Takeaways

Specialized Roles

Three-role mechanism.

Logic Fidelity

Micro Snippet Semantic Calculation (MSSC).

Readability

Variable renaming, appending comments, and structure simplification.

Human-Centric

Especially for newcomers.

Generalization

Decompiler-agnostic (Ghidra, IDA Pro).

Limitations of GPT & Direct Fine-Tuning

- **GPT limitations**

- Struggles with structured code transformation
- Limited understanding of low-level semantics

- **Direct Fine-Tuning (E2E)**

- Input: decompiled code
- Output: original source code
- Requires learning full transformation end-to-end
- Hard to generalize across patterns

```
1 unsigned long long sub_404056
2 (long long a1, unsigned short a2){
3     unsigned int16 v3;
4     unsigned int v4;
5     void *dest;
6     if(*(unsigned int *) (a1 + 28) == 1){
7         dest = *(void **) (a1 + 28);
8         // should be `8`
9         v4 = v3 * a2;
10        sub_406BB9(v4);
11    }
12 }
```

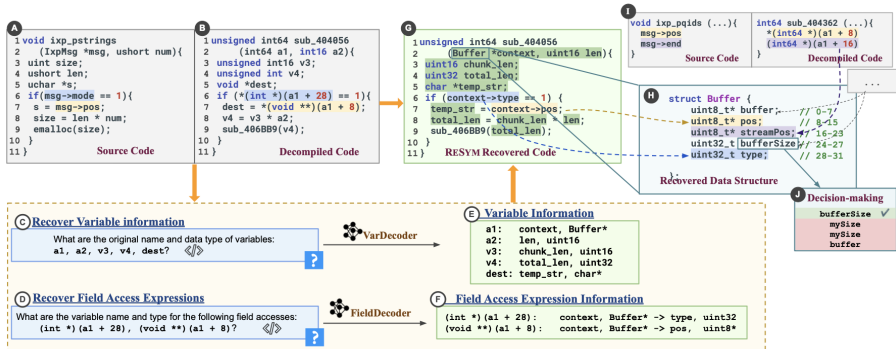
GPT-4 Output

```
1 void process
2 (state *s, unsigned short n){
3     unsigned short len;
4     int size;
5     char *tmp;
6     if (s->last == 1) {
7         tmp = s->tmp;
8         size = len * n;
9         min(len); // should be `size`
10    }
11 }
12 }
```

Fine-tuned E2E Model Output

ReSym: LLM-Powered Symbol Recovery [4]

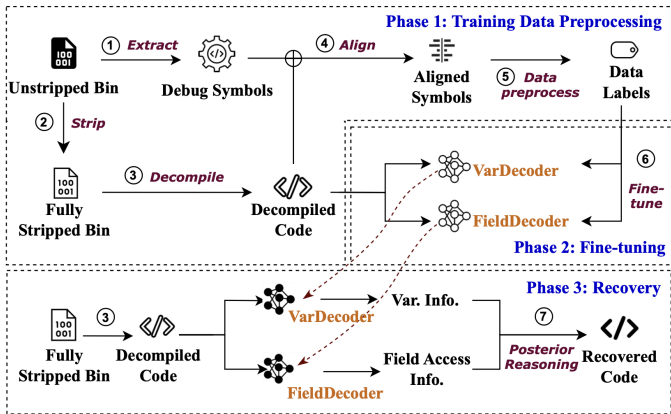
CCS 2024 🏆



Motivating Example

Core Innovation: The Hybrid Architecture

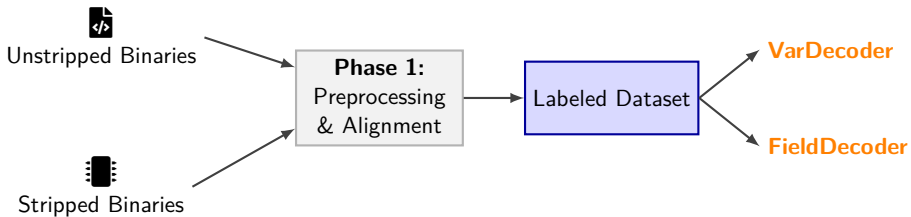
VarDecoder $\Rightarrow$ FieldDecoder $\Rightarrow$ Prolog Inference



Overview of Ground Truth Creation

Primary Objective:

Create accurately labeled datasets to fine-tune two core models.



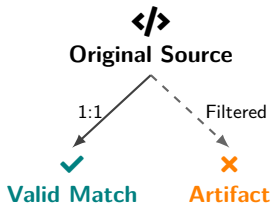
Methodology:

A hybrid approach aligning original source symbols to fragmented decompiled code.

VarDecoder: Variable Alignment Strategies

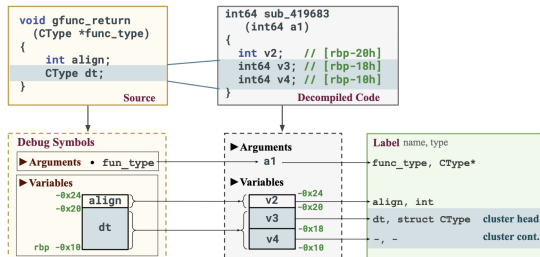
1. Exact Match (Singletons)

- **1:1 Mapping:** Fits within the memory bounds of the source variable.
- **Filtering:** Compiler artifacts are discarded.

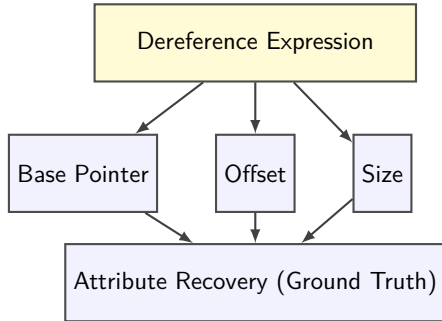


2. Fragmented Structures (Clusters)

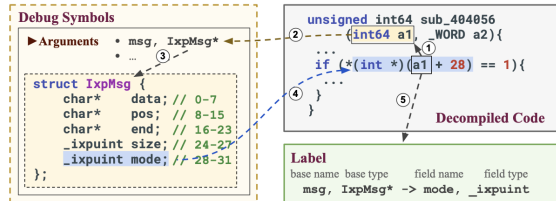
- **Problem:** Decompilers split structures (e.g., 16-byte struct → two 8-byte vars).
- **Labeling:** Cluster head kept.



FieldDecoder: Field Access Expression Labeling



Outputs: Base Name, Base Type, Field Name, Field Type



Dataset Scale and Sanitization

Metric	Detail
Source Scale	7,416 GitHub Projects (C/C++)
Binary Volume	113,696 Binary Files
Sanitization	Retained binaries $\geq$ 80% unique funcs

- **Crucial:** Splitting by project (rather than function) $\rightarrow$ avoid data leakage.

StarCoder 3B

Token Policy

- VarDecoder: >2K truncate ▪ >4K drop
- FieldDecoder: no truncate ▪ >4K drop
- Inference: 8K limit ▪ no drop

Training

- 4× A100 ▪ 3 epochs
- AdamW ▪ Batch 16
- LR $5e^{-5}$ + cosine decay

Fine-Tuning: VarDecoder

Prompt

“What are the original name and data type of variables: $v_1, \dots, v_m? < d >$ ”

Training Objective

$$\begin{aligned} L &= -\log P(Y_1 | I, \theta) \\ &= -\log \left[\underbrace{P(v_1 | I, \theta) + P(n_1 | I, v_1, \theta) + P(t_1 | I, v_1, n_1, \theta)}_{\text{Loss of recovering the first variable}} \right. \\ &\quad \left. -\log \sum_{i=2}^m \left[\underbrace{P(v_i | I, \{v_j, n_j, t_j\}_{j=1}^{i-1}, \theta)}_{\text{Conditioned on previous recovery}} \right. \right. \\ &\quad \left. \left. + P(n_i | I, \{v_j, n_j, t_j\}_{j=1}^{i-1}, v_i, \theta) \right. \right. \\ &\quad \left. \left. + P(t_i | I, \{v_j, n_j, t_j\}_{j=1}^{i-1}, v_i, n_i, \theta) \right] \right] \end{aligned} \quad \left. \vphantom{\sum_{i=2}^m} \right\} \text{Loss of recovering following variables}$$

Let $V_d = \{v_1, \dots, v_m\}$ where v_i is the i -th variable in the decompiled function d .

For each variable v_i :

- n_i is its original name
- t_i is its data type

Takeaway: Learns *structured, sequential* variable recovery from decompiled code

Fine-Tuning: FieldDecoder

Prompt

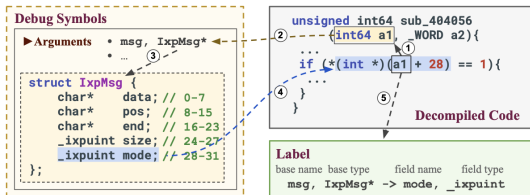
“What are the variable name and type for the following field accesses: $e_1, \dots, e_k$? $\langle d \rangle$ ”

Training Output

$$\begin{pmatrix} e_1 & nb_1 & tb_1 & \rightarrow & nf_1 & tf_1 \\ e_2 & nb_2 & tb_2 & \rightarrow & nf_2 & tf_2 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ e_k & nb_k & tb_k & \rightarrow & nf_k & tf_k \end{pmatrix}$$

For each field access expression e_i :

- nb_i is its base name
- tb_i is its base type
- nf_i is its field name
- tf_i is its field type



Phase 3: Posterior Reasoning Overview

Goal: Infer accurate, globally consistent variable types and names across functions.

The Inputs

- **Initial Facts:** From ML decoders and program syntax.
- **Derived Facts:** Generated via static reasoning rules.

Key Challenges

- Incomplete local information
- Cross-function dependencies
- Conflicting AI predictions

The Challenge: A Running Example

Decompiled Code Snippet

```
void sub_A(struct S* a1, int len) {  
    sub_B(a1);  
    memset(a1, 0, len);  
}
```

```
void sub_B(struct S* x) {  
    x->bufferSize = 128;  
}
```

Our Objective:

Infer the type, structure, and field names for `a1` despite limited local context in `sub_A`.

Step 1: Fact Extraction & Initialization

Before reasoning, we map the local landscape and initialize our groupings.

1. Extracting Local Facts

- **Variables:**

`Var(sub_A, a1) & Var(sub_B, x)`

- **Callsites:**

`Callsite(sub_A, sub_B, a1, 1)`

`Callsite(sub_A, memset, a1, 1)`

- **Field Access (in sub_B):**

`FieldAcc(x, off=0, sz=4, int,
bufferSize)`

2. Initializing Type Groups

- Each variable starts in isolation to prevent premature assumptions:

- $Tg_1 = \{a1\}$

- $Tg_2 = \{x\}$

Step 2: Inter-procedural Inference

We connect variables across function boundaries using behavioral rules.

Rule: Argument Passing Propagates Types

- `sub_A` calls `sub_B(a1)` $\Rightarrow$ `a1` corresponds to `x`.
- **Action:** Merge type groups $\Rightarrow Tg_1 = Tg_2 = \{a1, x\}$.

Rule: Isolate Type-Agnostic Arguments

- `sub_A` calls `memset(a1, 0, len)`.
- The first argument of `memset` is a generic pointer (`void*`).
- **Action:** Flag as `TypeAgnosticArg(memset, arg1)`. No type information is propagated through this call.

Step 3: Resolving Prediction Conflicts

The Problem: Candidates = bufferSize, mySize, buffer. A simple majority vote might pick a highly frequent, but incorrect, generic name.

Solution: Similarity-Biased Voting

1. Tokenize Candidates:

- bufferSize → (buffer, size)
- mySize → (my, size)
- buffer → (buffer)

2. Measure Overlap: bufferSize shares semantics with *both* alternatives.

3. Final Decision: Select bufferSize as the globally consistent field name.

Final Outcome

Inferred Structure

```
struct S {  
    int bufferSize;  
};
```

Resolved Variables

- `a1 : struct S*`
- `x : struct S*`

Key Insights:

- Structural and type information is successfully recovered across multiple, disconnected functions.
- Posterior reasoning safely bridges the gap between raw ML predictions and strict program semantics to resolve ambiguity.

Evaluation Metrics

- **VarDecoder**: Evaluates the recovery of variable information (names, types, and clustering).
- **FieldDecoder**: Evaluates the recovery of field access expressions.
- Assesses the capability to recover user-defined structures using posterior reasoning.

VarDecoder: Variable Name and Type Recovery

- Uses a **perfect match criterion** across all variables V in the test set.
- **Name Accuracy** (Acc_n):

$$Acc_n = \frac{Nm_c}{|V|}$$

(Nm_c = total number of correctly predicted names)

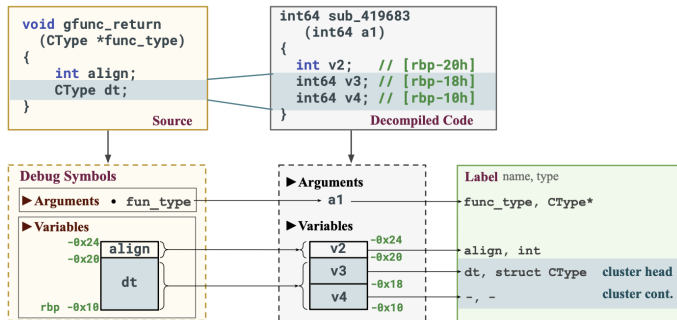
- **Type Accuracy** (Acc_t):

$$Acc_t = \frac{Ty_c}{|V|}$$

(Ty_c = total number of correctly predicted types)

VarDecoder: Variable Cluster Identification

- **Objective:** Identify stack-inlined structures and arrays.
- **Criterion:** A cluster is correct *if and only if* it predicts the exact same set of variables as the ground truth.



FieldDecoder: Field Access Expression Recovery

- Evaluates the prediction of four specific properties for each field access expression:
 1. Base name
 2. Base type
 3. Field name
 4. Field type

Evaluation Results

Overall		In Train		Not in Train	
Name	Type	Name	Type	Name	Type
56.4	64.6	88.2	89.5	37.5	49.9

Table 1: VarDecoder's accuracy (%) on variable name and type prediction.

Evaluation Results

Overall		In Train		Not in Train	
Name	Type	Name	Type	Name	Type
56.4	64.6	88.2	89.5	37.5	49.9

Table 1: VarDecoder's accuracy (%) on variable name and type prediction.

Overall				In Train				Not In Train			
Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f
54.4	53.8	55.2	55.7	89.0	87.5	89.0	89.0	34.0	33.9	35.3	36.0

Table 2: FieldDecoder's accuracy (%) on predicting base name, base type, field name, and field type.

Evaluation Results

Overall		In Train		Not in Train	
Name	Type	Name	Type	Name	Type
56.4	64.6	88.2	89.5	37.5	49.9

Table 1: VarDecoder’s accuracy (%) on variable name and type prediction.

Overall				In Train				Not In Train			
Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f
54.4	53.8	55.2	55.7	89.0	87.5	89.0	89.0	34.0	33.9	35.3	36.0

Table 2: FieldDecoder’s accuracy (%) on predicting base name, base type, field name, and field type.

Method	Name	Type
RESYM-VarDecoder	51.2	63.9
GPT-4	18.7	30.6

Table 3: Few-Shot Learning Comparison.

Evaluation Results

Overall		In Train		Not in Train	
Name	Type	Name	Type	Name	Type
56.4	64.6	88.2	89.5	37.5	49.9

Table 1: VarDecoder’s accuracy (%) on variable name and type prediction.

Method	Name	Type
ReSYM-VarDecoder	51.2	63.9
GPT-4	18.7	30.6

Table 3: Few-Shot Learning Comparison.

Overall				In Train				Not In Train			
Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f	Name _b	Type _b	Name _f	Type _f
54.4	53.8	55.2	55.7	89.0	87.5	89.0	89.0	34.0	33.9	35.3	36.0

Table 2: FieldDecoder’s accuracy (%) on predicting base name, base type, field name, and field type.

Method	Struct Layout			Struct Annotation (Accuracy)		
	Precision	Recall	F1	Struct Type	Field Name	Field Type
ReSYM	72.9	28.9	41.4	46.8	16.3	16.9
ReSYM _{Light}	74.9	16.8	27.4	42.6	9.2	9.5

Table 4: Posterior Reasoning Results.

ReSym: Key Takeaways

Hybrid Logic

Merges the generative power of LLMs with deterministic program analysis.

Task Division

Breaks complex engineering down into sub-problems just like human experts.

Conflict Resolution

Similarity-biased voting guarantees the consistency of overarching structures.

Open Dataset

Provided an open large-scale dataset featuring over 113k binaries to aid future reverse engineering ML tasks.

The Core Question

Is the process of AI-driven enhancement in RE actually effective?

RQ1: Usage

How do practitioners use LLMs for Reverse Engineering?

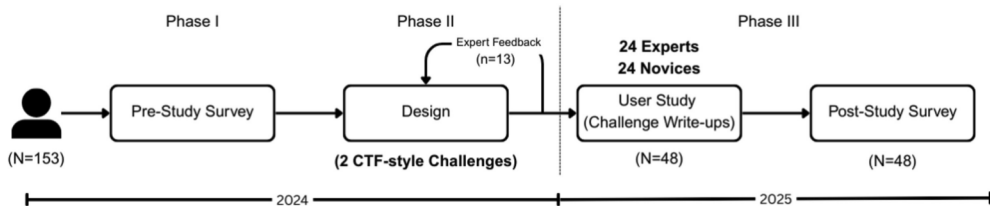
RQ2: Impact

How do LLMs impact overall RE performance and success?

RQ3: Collaboration

Do specific LLM collaboration patterns change RE outcomes?

Study Design & Scale



 **48 Participants**
24 Experts | 24 Novices

 **109 Hours**
Recorded RE Sessions

 **1,517 LLM Queries**
Experts: 728 | Novices: 789

From *Decompiling the Synergy* [1] - NDSS 2026 

Results: The Expertise Gap



Novices: Catching Up

- **Expert-Level Output:** Approached expert levels of reversing.
- **2x Improvement:** Doubled understanding rates using LLMs.
- **Key Driver:** Highly effective at *function summarization*.



Experts: Hitting a Plateau

- **Negligible Gains:** Almost no reversing improvements with LLMs.
- **Equal Usage, Less Benefit:** Made roughly the same queries as novices.
- **Takeaway:** Experts require a fundamentally different collaboration model.

Results: Collaboration Patterns & Timing



Wasted Collaboration

Over-collaboration harms output.



First Look Prevails

Initial impressions matter most.

Results: Hallucinations Remain a Severe Risk

⚠ A Rare But Catastrophic Failure Mode

Out of **1,517** LLM Responses, only **3** were detected as hallucinations ($< 1\%$).



3 Vulnerability Hallucinations

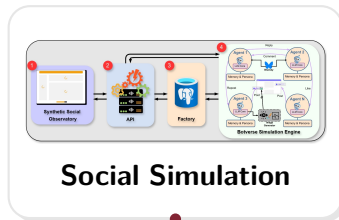
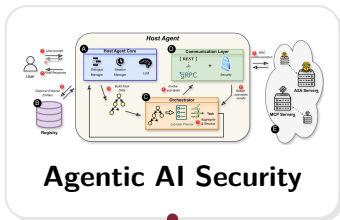
- **Massive Slowdowns:** Caused a minimum **231% slowdown** in code understanding rates.
- **Premature Failures:** *All* participants who encountered these hallucinations submitted an incorrect solution early.
- **Trust Issue:** Users over-relied on the AI's confident (but false) identification of vulnerabilities.

Summary: Is AI-Driven RE Effective?

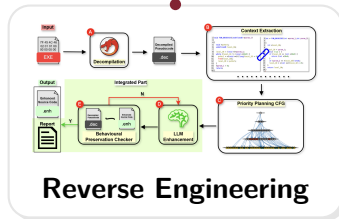
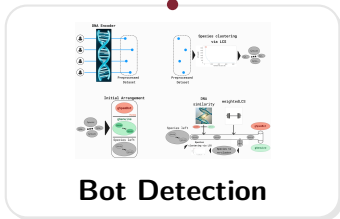
Yes, but with heavy caveats.

- ✓ **Levels the playing field** by significantly improving novice performance.
- ⊘ **Fails to consistently elevate experts** under current collaboration models.
- ⌚ **Speed is critical:** Benefits appear within seconds; prolonged interaction may reduce understanding.
- ☢ **High-stakes risk:** Even $< 1\%$ hallucination rates can critically derail the RE process.

Like This Stuff? Let's Collaborate



allegrini@di.uniroma1.it



References

- [1] Zion Leonahenahe Basque et al. “Decompiling the Synergy: An Empirical Study of Human–LLM Teaming in Software Reverse Engineering”. In: *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*. 2026.
- [2] Qibin Chen et al. “Augmenting Decompiler Output with Learned Variable Names and Types”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022.
- [3] Peiwei Hu, Ruigang Liang, and Kai Chen. “DeGPT: Optimizing Decompiler Output with LLM”. In: *Proceedings 2024 Network and Distributed System Security Symposium (NDSS)*. 2024.
- [4] Danning Xie et al. “ReSym: Harnessing LLMs to Recover Variable and Data Structure Symbols from Stripped Binaries”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024.